

Gestion des dépendances, risques et maintenabilité

Jour 1 - 18/09/26

Qui suis-je ?

Philippe AUSSEL

- Début de codeur à 14 ans sur CPC 464
- 10 ans en tant que développeur
- 18 ans en tant qu'architecte/manager/CTO
- Mais toujours développeur dans l'âme



Programme

- Jour 1 — Comprendre et maîtriser ses dépendances.
- Jour 2 — Découpler, packager et sécuriser ses choix.
- Jour 3 — Appliquer, puis évaluer.

JOUR 1 — Comprendre et maîtriser ses dépendances.



Partie 1 : C'est quoi une dépendance ?

Définition

Une dépendance est tout élément dont un module a besoin pour fonctionner. Cela peut être une fonction, une classe, une bibliothèque, un service externe... Sans cette dépendance, le module ne peut pas exécuter sa tâche.

“A dépend de B” signifie : si B change, A peut casser.

Des dépendances, il y en a partout

- Une librairie, un framework, une package
- Une API, un service cloud, ..
- Une base de données, un partage de fichier
- Un protocole de communication

Dépendances internes

- Un module Front qui dépend d'un module Services
- Une classe qui dépend d'une autre classe
- Un projet partagé (Common, Shared...)

Dépendances externes

- Librairies : NuGet, npm, pip...
- Frameworks
- API tierces
- Bases de données

Les externes sont souvent plus risquées : on les maîtrise moins.

Directes ou transitives

Directes

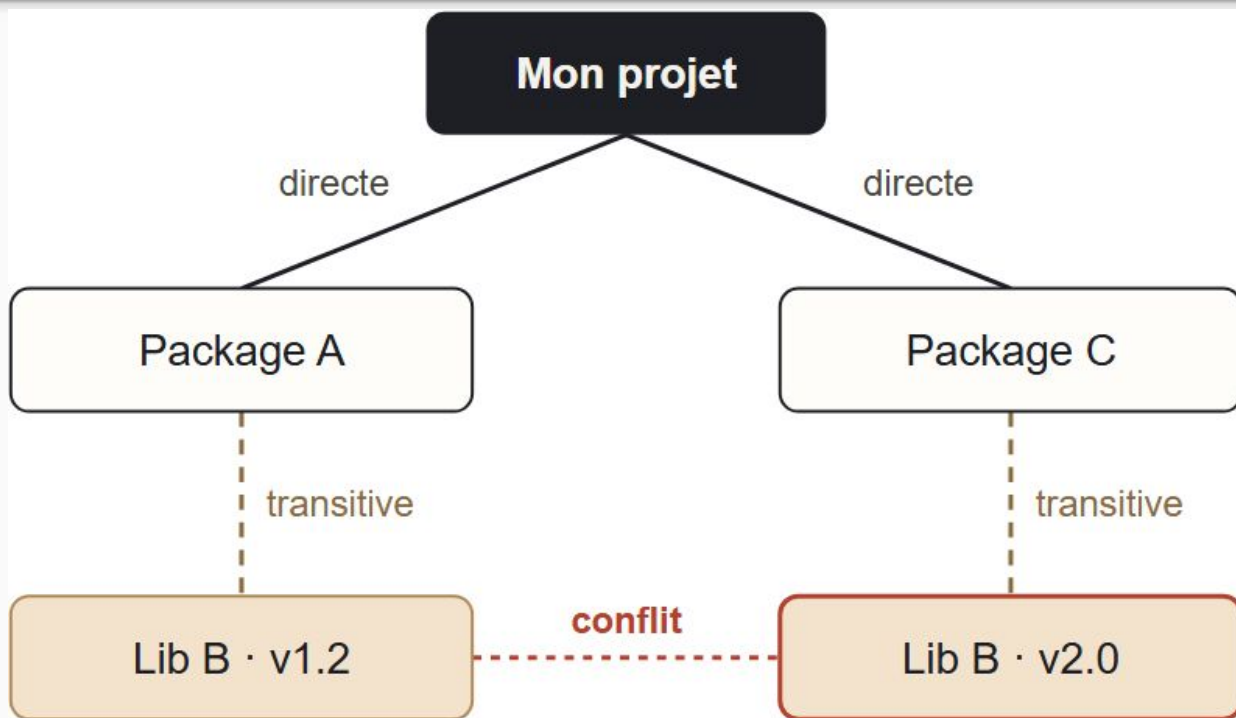
Vous l'appellez explicitement dans votre code.

Transitives

Jamais mentionnée : elle arrive avec une autre. Votre projet utilise A, A utilise B, donc votre projet dépend aussi de B.

Invisibles, elles sont souvent sources de bugs.

Directes ou transitives



Explicites ou implicites

Explicites : Visibles dans le code

- import, using, #include
- new MaClasse()
- Un appel d'API écrit en clair

Implicites : Rarement documentées

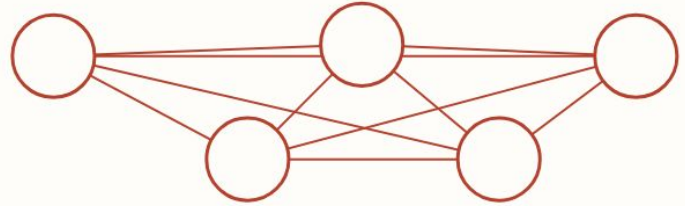
- Variables d'environnement
- Fichiers de configuration
- Format de données attendu

Couplage



Couplage faible : c'est bon

Un changement dans un module casse peu le reste.



Couplage fort : danger

Effet domino, maintenance difficile.

L'objectif : minimiser le couplage, pas éliminer les dépendances.

Cohésion

Faible cohésion : Un module qui fait tout et n'importe quoi.

```
class ClientManager
{
    bool ValiderSiret(string s);
    void EnvoyerEmail(Client c);
    byte[] FacturePdf(Facture f);
    void ExporterCsv(Client[] l);
}
```

Haute cohésion : Des modules clairs, stables, faciles à maintenir.

```
class SiretValidator { ... }

class ClientMailer { ... }

class InvoicePdfGenerator { ... }

class ClientCsvExporter { ... }
```

Couplage faible + cohésion forte = architecture saine

	Couplage faible	Couplage fort
Cohésion forte	Architecture saine, modules clairs, changements localisés.	Modules rigides, chacun est clair, mais tout bouge ensemble.
Cohésion faible	Code éparpillé et Indépendant, mais responsabilités diluées.	Tout dépend de tout, rien n'est clair.

Pourquoi les dépendances fragilisent un projet

- **Propagation** : Si la dépendance casse, tout ce qui repose dessus casse aussi.
- **Mise à jour** : Une nouvelle version introduit une incompatibilité.
- **Disponibilité** : L'API externe est lente ou indisponible.
- **Sécurité** : Une vulnérabilité est découverte dans un package.
- **Humain** : Une seule personne comprend la dépendance.

Une dépendance mal gérée = dette technique + instabilité.

Quand une dépendance devient-elle un problème ?

- **Elle change souvent** : API instable, ruptures de compatibilité fréquentes
- **Plus maintenue** : Ni correctifs de bugs, ni correctifs de sécurité
- **Fortement couplée** : Ses types et ses appels sont partout dans votre code
- **Utilisée partout** : Un SPOF en puissance
- **Non isolée** : Aucune interface entre elle et le métier
- **Licence risquée** : GPL ou AGPL dans un produit propriétaire

Le problème n'est pas la dépendance : c'est l'absence de contrôle.

Une dépendance n'est pas que du code

- **Vendor lock-in (Fournisseur)** — Sortir coûte cher : formats fermés, services managés, compétences spécifiques
- **Pricing (Prix)** — Le modèle tarifaire ou la licence peuvent changer du jour au lendemain
- **Souveraineté (Juridiction)** — Où sont les données, et quel droit s'applique à l'opérateur ?
- **Bus factor (Personnes)** — Un mainteneur unique, une compétence rare dans l'équipe

Approfondi durant le jour 2

Exercice

Inventoriez les dépendances d'un projet que vous connaissez

- Listez au moins 10 dépendances : code, infrastructure, services, configuration.
- Classez-les : interne ou externe, directe ou transitive, explicite ou implicite.
- Notez leur couplage avec votre code : faible, moyen ou fort.
- Entourez celle qui vous inquiète le plus, et dites pourquoi.

A retenir

- Une dépendance est tout ce dont votre code a besoin, et pas seulement du code.
- Moins vous contrôlez une dépendance, plus elle est risquée.
- Transitives et implicites sont les plus dangereuses : on ne les voit pas.
- Visez un couplage faible et une cohésion forte.

Partie 2 : Identifier les dépendances dans un projet



Quatre endroits où chercher

- **Le code** : import, using, #include, require()
- **Le réseau** : appels HTTP, SDK cloud, files de messages
- **Les manifestes** : .csproj, package.json, requirements.txt
- **Les ressources** : bases, stockage, cache, secrets

Puis ce qui ne se voit nulle part : les dépendances cachées.

Méthode 1 : Repérer les imports

Comment faire : Parcourir les fichiers source ou utiliser un outil d'analyse statique pour ces mots-clés.

Ce que ça révèle : Les dépendances directes et explicites de chaque fichier.

Premier réflexe d'analyse : lire la liste des imports.

```
// C#  
using MyProject.Data;  
using Newtonsoft.Json;  
  
// JavaScript  
import express from 'express';  
  
// Python  
from pandas import DataFrame  
  
// C / C++  
#include <curl/curl.h>
```

Méthode 2 : Traquer les appels réseau

Ce qu'on cherche

- API REST et GraphQL
- SDK cloud (Azure, AWS...)
- Microservices internes et externes
- Files de messages : Kafka, RabbitMQ

Comment : Rechercher fetch, axios.get, HttpClient.SendAsync... ou analyser les logs de trafic sortant.

```
var url = "https://api.open-  
meteo.com/v1/forecast"  
+ $"?latitude={lat}&longitude={lon}";  
  
var response = await _http.GetAsync(url);  
response.EnsureSuccessStatusCode();
```

Méthode 3 : Lire les manifestes

.NET	*.csproj	packages.lock.json	NuGet
JavaScript	package.json	package-lock.json	npm, Yarn
Python	requirements.txt, pyproject.toml	poetry.lock	pip, Poetry
Java	pom.xml, build.gradle		Maven, Gradle
PHP	composer.json	composer.lock	Composer

Méthode 4 : Suivre l'accès aux ressources

- Bases de données SQL et NoSQL
- Systèmes de fichiers
- Stockage objet : S3, Azure Blob
- Caches : Redis
- Secrets et identité : Key Vault, annuaire

Chercher les chaînes de connexion, les configurations d'accès et les drivers ou clients utilisés.

```
appsettings.json
{
  "ConnectionStrings": {
    "Crm": "Server=sql01;Database=Crm;..."
  },
  "Storage": {
    "Exports":
    "https://acme.blob.core.windows.net"
  },
  "Redis": "cache01:6379"
}
```

Les dépendances cachées

- **Configuration et environnement** : `process.env.DB_HOST`, fichiers YAML ou JSON qui changent le comportement
- **Singletons et état global** : Une instance unique et globale : couplage fort, impossible à remplacer en test
- **Outils externes** : Le code lance un binaire (ffmpeg, gzip, un compilateur) qu'il ne gère pas
- **Contexte d'exécution** : Horloge, fuseau horaire, culture (séparateur décimal), système d'exploitation

Source majeure de dette technique et de régressions surprises.

Combien de dépendances dans cette méthode ?

```
public void Export()
{
    var host = Environment.GetEnvironmentVariable("DB_HOST");
    var rows = Database.Instance.Query("SELECT * FROM Clients");
    var file = $"C:\\exports\\clients_{DateTime.Now:yyyyMMdd}.csv";
    File.WriteAllText(file, ToCsv(rows));
    Process.Start("gzip", file);
}
```

Au moins 8

1. Variable d'environnement DB_HOST
2. Singleton Database.Instance
3. Schéma de la table Clients
4. Chemin Windows C:\\exports
5. Horloge système
6. Système de fichiers
7. Binaire gzip installé
8. Culture utilisée par ToCsv

Les outils pour lister et auditer

.NET	dotnet list package --include-transitive	dotnet list package --vulnerable dotnet list package --outdated
JavaScript	npm ls --all	npm audit npm outdated
Python	pipdeptree	pip-audit pip list --outdated
Graphe de code	NDepend (.NET) · dependency-cruiser, madge (JS)	Visualiser le couplage et les cycles

Cartographier : de la liste au graphe

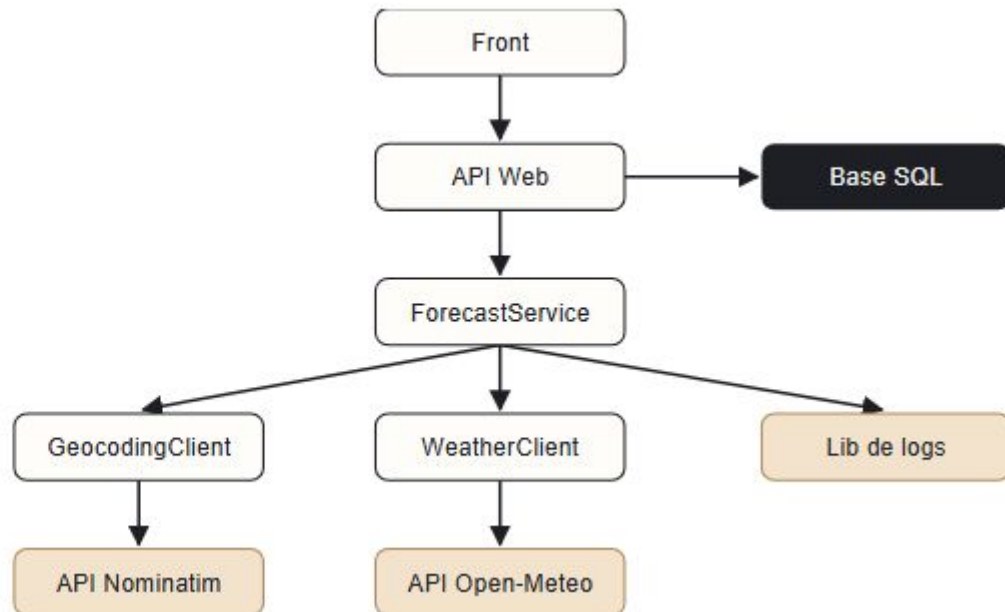
Un listing ne montre pas les interconnexions. Le graphe donne une vue **macro** (qui dépend de qui) et **micro** (quelles classes).

☐ Module interne

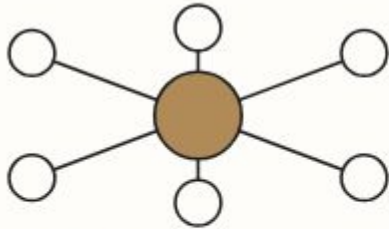
☒ Externe : package ou API

☒ Infrastructure

— « dépend de »

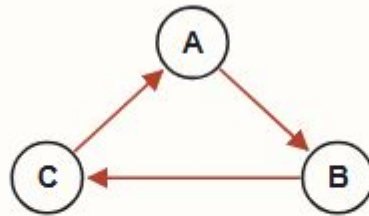


Ce que le graphe révèle



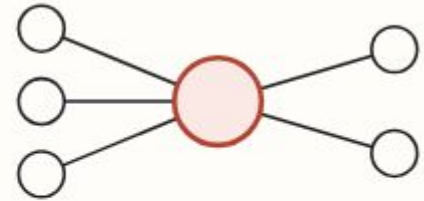
Couplage fort

Un nœud relié à beaucoup d'autres :
chaque changement s'y propage.



Dépendances circulaires

$A \rightarrow B \rightarrow C \rightarrow A$: difficile à tester et à décomposer.



Chemins critiques et SPOF

Le point par où transite la majorité
des flux.

Détecter les dépendances critiques

Les 3 critères :

- **Fréquence** : Combien de modules l'utilisent ?
- **Rôle métier** : Est-elle indispensable au cœur de métier ?
- **Contrôle** : Pouvez-vous la corriger ou la remplacer ?

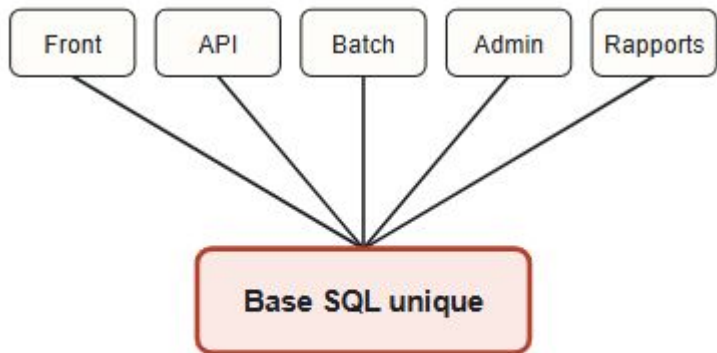
SPOF : Single Point of Failure

Exemples classiques

- Une base de données unique
- Un service d'authentification centralisé
- Une librairie de logs utilisée par 100 % des modules

Stratégies

- Isoler derrière une interface
- Dédoubler : réplication, failover
- Circuit breaker et cache pour un mode dégradé



Elle tombe : tout s'arrête

A retenir

- Imports et manifestes révèlent les dépendances directes et les librairies externes.
- Appels réseau et ressources montrent ce que vous ne contrôlez pas..
- Configuration, singletons et contexte d'exécution cachent des dépendances..
- Le graphe rend visibles couplage, cycles, dépendances critiques et SPOF..

Partie 3 : Architecture, IoC et injection de dépendances ?



L'architecture décide de vos dépendances

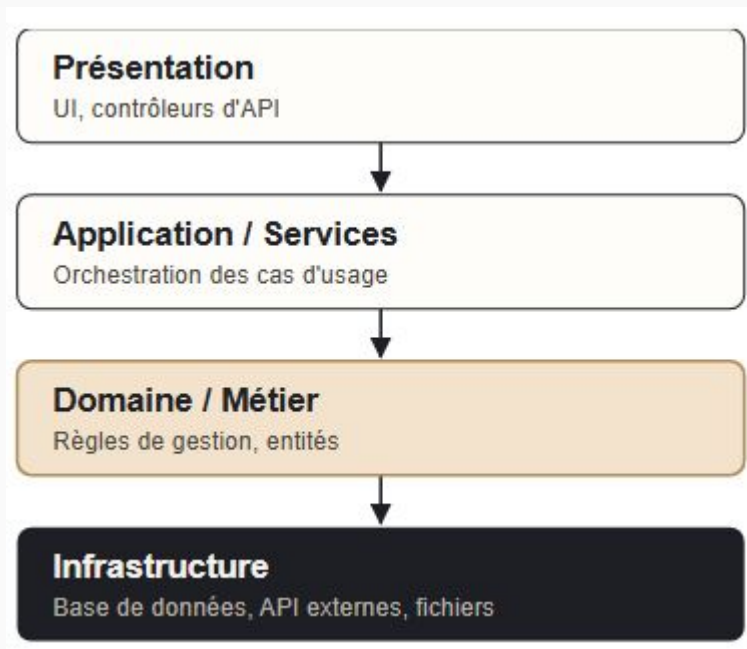
Selon l'organisation des modules, on peut :

- réduire ou augmenter le couplage
- faciliter ou compliquer la maintenance
- isoler ou propager les dépendances externes
- rendre le code testable... ou non
- limiter les effets domino d'un changement

Une bonne architecture
maîtrise les dépendances.

Une mauvaise architecture
les subit.

L'architecture en couches



Objectifs :

- Séparer les responsabilités
- Protéger le métier de la technique
- Limiter la propagation des dépendances externes
- Simplifier les tests
- Isoler base et API externes

Chaque couche dépend de celle du dessous, jamais de celle du dessus.

Le problème du new partout

```
public class OrderService
{
    public void Confirm(Order order)
    {
        var email = new SmtplibEmailService();
        email.Send(order.CustomerEmail,
            "Commande confirmée");
    }
}
```

- **Fortement couplé** : Lié à SmtplibEmailService pour toujours.
- **Impossible à tester** : Chaque test envoie un vrai e-mail.
- **Fragile** : Si le constructeur change, l'appelant casse.
- **Rigide** : Passer au SMS impose de modifier la classe.

Le code choisit lui-même ses dépendances : c'est ce que l'IoC évite.

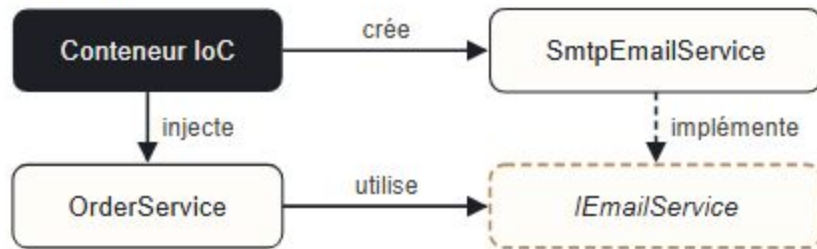
Inversion de contrôle : ce n'est plus le module qui va chercher ses dépendances

Sans IoC



Le service crée et choisit sa dépendance

Avec IoC



- **Réutilisable** : Le module marche avec toute implémentation.
- **Testable** : On lui fournit un fake en test.
- **Remplaçable** : On change la dépendance sans toucher au module.

L'injection de dépendances : trois formes

RECOMMANDÉE

Par constructeur

Claire, obligatoire, testable.

```
public OrderService(  
    IPaymentGateway gateway)  
{ _gateway = gateway; }
```

OPTIONNELLE

Par propriété

Moins stricte : peut rester vide.

```
public ILogger? Logger  
{ get; set; }
```

PONCTUELLE

Par méthode

Pour un besoin lié à un appel.

```
public void Export(  
    IExportFormat format)  
{ ... }
```

DI : le module ne crée plus ses dépendances, il les reçoit.

Le conteneur IoC fait le travail

Ce que fait le conteneur :

- Il crée les objets
- Il gère leur durée de vie
- Il résout les dépendances en cascade
- Il permet de remplacer une implémentation en un point

.NET	Microsoft.Extensions. DependencyInjection
Java	Spring
Node.js	InversifyJS
Python	dependency-injector

Durées de vie : Transient, Scoped, Singleton

AddTransient	à chaque résolution	Services légers, sans état
AddScoped	par requête HTTP	DbContext, unité de travail
AddSingleton	pour toute l'application	Cache, configuration

Piège : la dépendance captive

Un singleton qui reçoit un service scoped le garde pour toujours : état partagé entre requêtes.

Pourquoi l'IoC rend le code testable

Avant

```
var repo = new SqlUserRepository();  
var service = new UserService(repo);  
  
// Impossible de tester  
// sans base SQL disponible
```

Avec IoC

```
class FakeUserRepository : IUserRepository  
{  
    public User? Find(string email)  
        ⇒ new User(email);  
}  
  
var service = new UserService(  
    new FakeUserRepository());  
Assert.True(service.Exists("a@b.fr"));
```

Tests unitaires simples - Aucune dépendance lourde - Aucun effet de bord - Tests rapides

A retenir

- L'architecture en couches limite la propagation des dépendances.
- Le new dispersé crée un couplage fort et un code instable.
- IoC : le module ne choisit plus ses dépendances. La DI les lui fournit.
- Le conteneur gère création et durée de vie ; les fakes rendent les tests simples.

Partie 4 : TP1 : API Météo ?

L'objectif

Construire une API qui reçoit une adresse postale en GET et renvoie les prévisions météo du lieu, avec les bonnes pratiques vues cet après-midi : couplage faible, IoC, DI.

Requête

```
GET /forecast?address=Alès
```

Réponse attendue

```
{  
  "address": "Alès",  
  "latitude": 44.13, "longitude": 4.08,  
  "hourly": { "shortwave_radiation": [ ... ] }  
}
```

Deux services externes à enchaîner

- **GEOCODING : Nominatim (Nom de lieu → latitude / longitude)**

nominatim.openstreetmap.org/search?q=Alès&format=json

- **MÉTÉO : Open-Meteo (Latitude / longitude → prévisions)**

api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35&hourly=shortwave_radiation